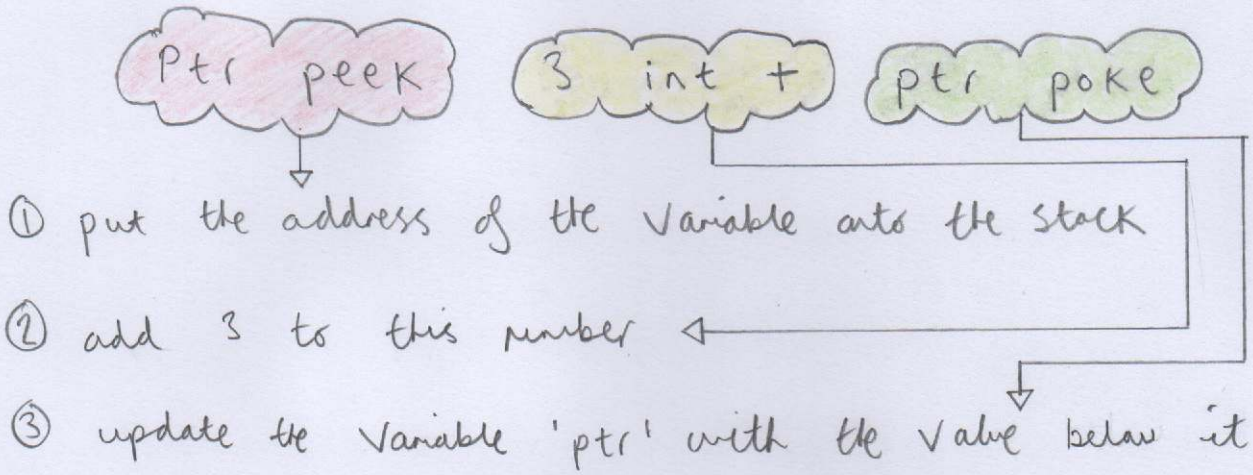
POINTERS, MEMORY ALLOCATION & USER DEFINED DATA STRUCTURES

The C language and Assembly language both use pointers to access memory. This language uses 8 bytes for an integer, that means we can use this data type as a pointer too, 8 bytes is enough to store any memory address.

" a ▽ short ▽ string " ptr def-i

Here we can treat ptr variable as a pointer, because it is holding the memory addresses of the string in speech marks.

To move the pointer +/- a number of bytes, that's easy to do:



Now the stack will be empty of these values and the ptr variable will now point to the 'h' character in the word 'short'.

Using pointers this way, we can easily allocate memory on the heap! The pointer / integer can handle the allocation. Let's say we needed 40 bytes on the heap, we can easily do this by writing this code:

40 int mem-alloc ptr-data def-i

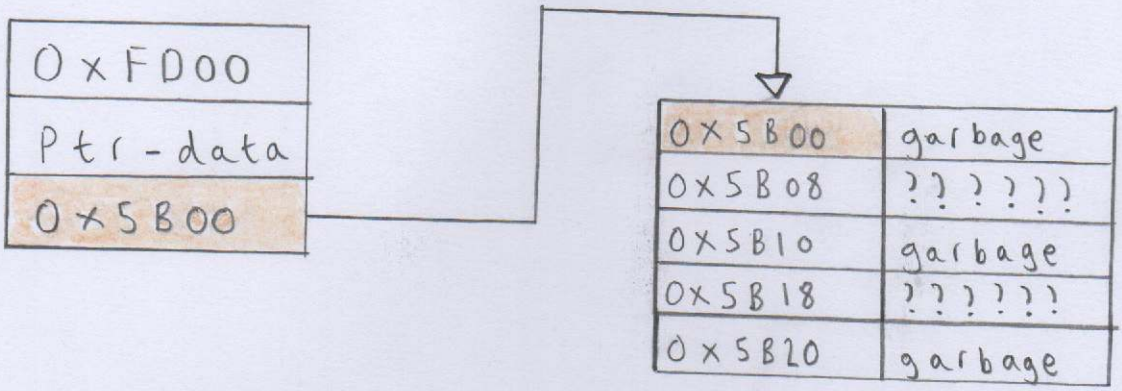
① mem-alloc looks at the number on the top of the RTS and allocates this number of bytes using a library call 'malloc'.

② The address of the heap allocation is left on top of the RTS

③ def-i creates a new variable 'ptr-data' and stores the address from #2 in this variable

Now that ptr-data has the address of the heap alloc of 40 bytes, we can treat it the same way as a structure. That means we can read data (using peek) and write data (using poke).

After mem-alloc was called, we have this:

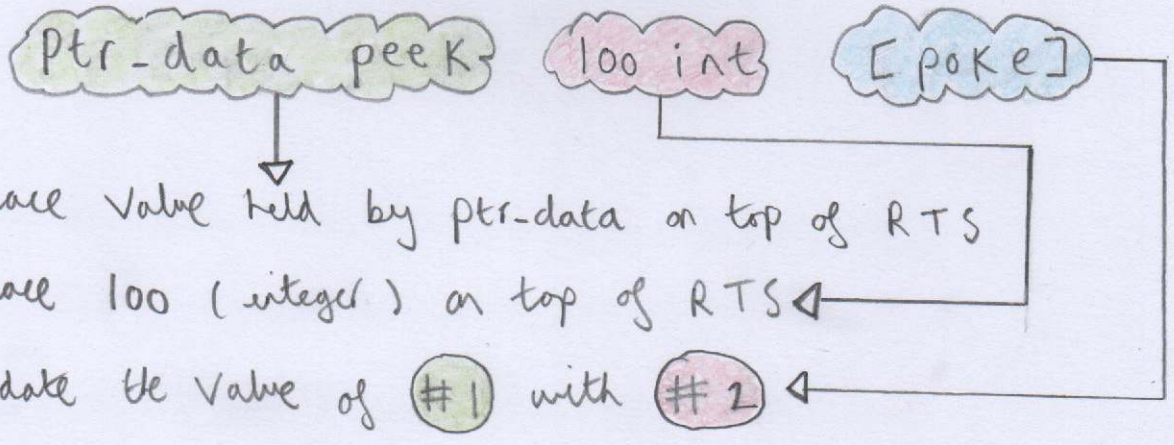


To create two strings, we can use:

"cats ▽ and ▽ dogs" alpha-1 def-i

"World ▽ Cup ▽ heroes!" alpha-2 def-i

Now we can place values directly into the 40 bytes on the heap, just like a struct in 'c'.



We can update the rest of this heap allocation by doing more updates:

```
Ptr_data peek 8 int + 5000 int [poke]
Ptr_data peek 16 int + alpha-1 peek [poke]
Ptr_data peek 24 int + alpha-2 peek [poke]
ptr-data peek 32 int + -321 int [poke]
```

Let's check the heap allocation now:

Heap Alloc

0x5B00	100
0x5B08	5000
0x5B10	0x33D0
0x5B18	0x35D0
0x5B20	-321

0x33D0	1	2	3	4	5	6	7	8	9	A	B	C	D
C	a	t	s	▽	a	n	d	▽	d	o	g	s	10

0x35D0	D1	D2	D3	D4	D5	D6	D7	D8	D9	DA	DB	DC	DD	DE	DF	E0	E1
W	o	r	l	d	▽	C	u	p	▽	h	e	r	o	e	s	!	10

To extract values from the memory allocated on the heap, it's quite simple.

For example, to get to the string in address 0x5B18, first get the ptr-data value and put it on the top of the RTS.

```
Ptr_data peek
```

Now move 24 bytes forward:

```
24 int +
```

To place the value at this address on the top of the RTS we use [peek]. The code to do this is simple:

```
Ptr_data peek 24 int + [peek]
```

Now we have the pointer address to "world ▽ cup ▽ heroes!" on the top of the run time stack!!

To get the value at memory location 0x5B20, it's the same but the offset is 32 bytes ahead of 0x5B00.

```
Ptr-data peek 32 int + [peek]
```

Using [peek] and [poke] to read/write memory locations is useful, but we have big problem when we do not wish to read/write 8 bytes at a time!

Let's say we wanted to replace all the '0' chars with an Ø (Zero) in the string "World ▽ Cup ▽ Heroes!" then the [poke] operator would alter 8 bytes, not 1 byte! This would ruin the string!

Because of this we added some operators that would allow us to read/write to memory on the heap alloc 1, 2, 4 and 8 bytes.

Here is the equivalent C code for the previous example:

```
const char *alpha-1 = "cats ▽ and ▽ dogs.",
           *alpha-2 = "World ▽ Cup ▽ Heroes!";

Void * data-ptr = malloc(40);

*(int *) data-ptr = 100;
*(int*) (data-ptr +4) = 5000;
*((const char **) (data-ptr+8) = alpha-1;
*((const char **) (data-ptr +16) = alpha-2;
*(int *) (data-ptr + 24) = -321;
```

Mistake! in the previous page DEC/2025

The last 'C' example was incorrect because "alpha-1" and "alpha-2" are both in the pool of constant strings that remain in .rodata.

Reading them is fine, but writing/updating them is impossible.

Here is the correction:

```
char alpha-1[]="cats and dogs";
char alpha-2[]="World Cup Heroes!";

*(char**)(data-ptr+8) = alpha-1;
*(char**)(data-ptr+16) = alpha-2;
```

Now we can read & write to both strings alpha-1 & alpha-2, using data-ptr.

Lets change alpha-2[1] from 'o' to 'ø' using data-ptr:

```
*(*(char**)(data-ptr+16)+1)='ø';
```

To read memory we have byte-at, word-at, dword-at and qword-at. Now we can read bytes, words, double words and quad words.

To use any of these operators we just put the pointer on top of the stack and these operators will read 1, 2, 4 or 8 bytes from this pointer and put the value on top of the RTS.

```
"abcdefg" 3 int + word-at
```

Here the memory addresses of the string is put onto the stack. We move 3 bytes forward, now the pointer has moved from the 'a' to the 'd'. Word-at reads 2 bytes 0x64 and 0x65 and puts them on top of the RTS. If we use the print-ptr operator, we get the value 0x6564, print-i gives us the decimal 25956.

To write to a memory location, we have these operators.

```
upd-byte (for 1 byte)
upd-word ("2")
upd-dword ("4")
upd-qword ("8")
```

Here is an example:

```
"abcdefg" 22872 int 2 int upd-word
```

① put pointer to memory on top of RTS.

② put the value you want to change on top of RTS ("XY" in hex 0x5958, in decimal 22872.

③ how many bytes from pointer in #1 do we move forward? in this case it is 2 bytes.

Operator upd-word now updates the string to "abXYefg".

I enjoyed using these operators, because now we can update the memory like we do in 'C' and assembly language!

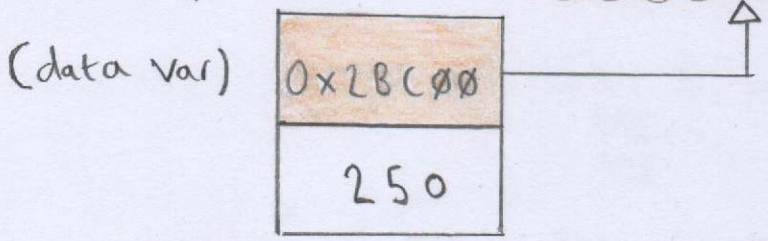
How do we pass memory locations to other functions? We simply push the address onto the RTS and once the function has been called, it can now grab this value.

```
func:   quadruple Value
        dup [peek] 4 int * [poke]
        return

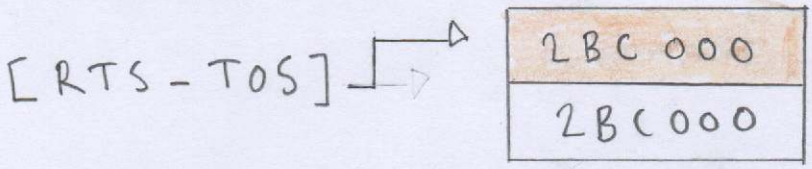
func:   double Value
        Ptr_1 def-i
        Ptr_1 peek dup [peek] 2 int * [poke]
        return

-start:
        8 int mem-alloc data def-i
        data peek 250 int [poke]
        data peek double Value call-func
        "--- after double Value ---" print-str pop
        data peek [peek] print-i pop
        data peek quadruple Value call-func
        data peek [peek] print-i pop
```

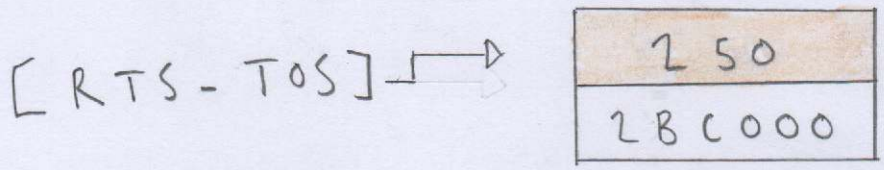
Function double Value declares a local var ptr_1 that has the memory address for the heap allocation. We peek this value and duplicate it, so that the data var looks like this (assuming our heap address is 0x2BC00).



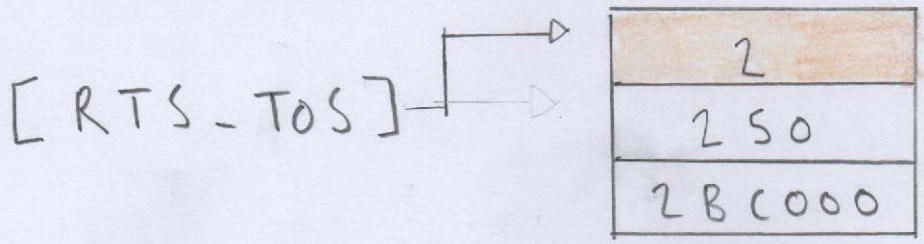
Once we're inside doubleValue function, the RTS looks like this after we peek the local var ptr-1 and then 'dup' it.



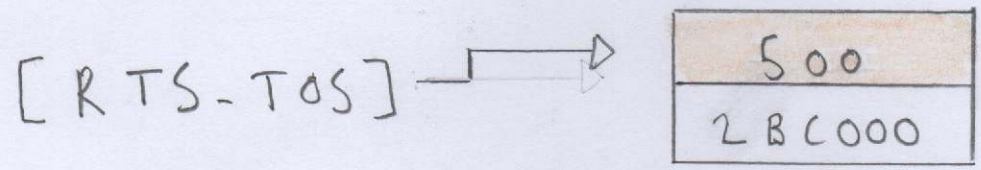
After we [peek] the RTS, we get the value at the address on the top of the RTS.



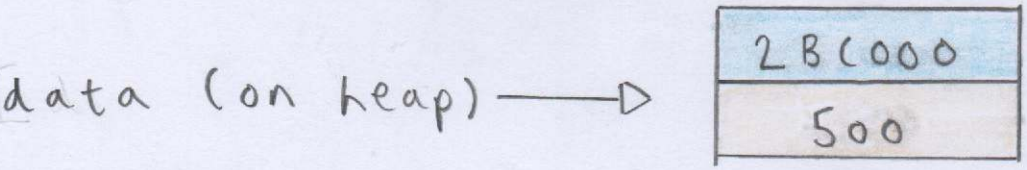
Now before we double this value we have:



After the '*' operator we have:



[poke] will now update the value at memory address 0x2BC000 with 500. Now the heap allocation looks like this:



Looking at function quadruple value, the logic is the same but here we aren't using a local variable to grab the pointer address.

Returning heap allocation is simple to do. Here's a simple example:

```
func:   heap-allocation
        8 int mem-alloc dup
        100 int rand ;( gereate random number < 100)
        [poke]
        return

-start:
        heap-allocation call-func ptr-1 def-i
```

In the heap-allocation function, we duplicate the address of the 8 bytes. The top most address is needed after we update the memory with a random number. The [poke] operator consumes this memory address and the value we're updating the memory with. That leaves the other copy of the memory address on top of the RTS.

After the function returns to -start, we grab this address in our GLOBAL variable Ptr-1.

Once we have allocated memory on the heap, we can use this to create our very own structures. We can create several structures that are different and combine the memory address of each structure into 1 structure. This will then replicate a struct in 'C' that has several different structures as structure members.

```
Struct  student {
        struct  courses  c1  [10];
        struct  attendance  attend;
        struct  personal-details  info;
        struct  form-tutor  class;
        struct  exam-details  exams  [50];
} a-student;
```

Later we will see code with linked lists, arrays, binary trees and hash sets.